U.E 5.1 - Intelligence Artificielle : Machine learning
Promotion FISE 2027

ENSTA BRETAGNE
2 Rue François Verny
29200 Brest Cedex 9, France

BARROS ALVES Luiz Felipe
luiz.barros@ensta.fr
BENIGNO DA SILVA Matheus Henrique
matheus.benigno@ensta.fr

Sommaire

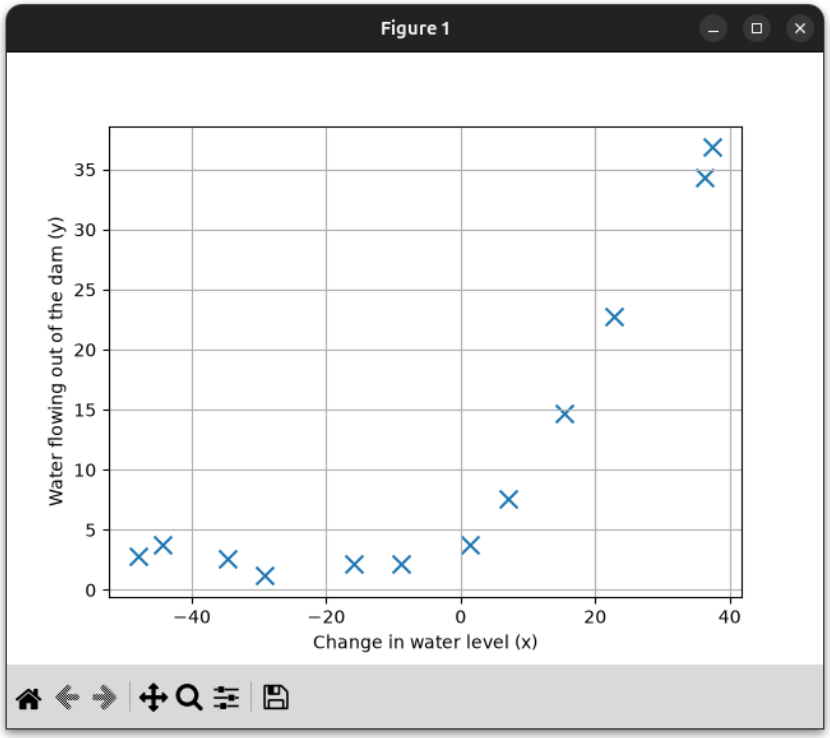
1 | 3 Regularized Linear Regression **2**
1.1 | 3.1 Visualizing the dataset . . . 2
1.2 | 3.2 Regularized linear regression cost function . . . 2
1.3 | 3.3 Regularized linear regression gradient . . . 3
1.4 | 3.4 Fitting linear regression . . . 4

2 | 4 Bias-variance **7**
2.1 | 4.1 Learning curves . . . 7

3 | 5 Polynomial regression **8**
3.1 | 5.1 Learning Polynomial Regression . . . 9
3.2 | 5.2 Adjusting the regularization hyper-parameter . . . 10
3.3 | 5.3 Selecting λ using a validation set . . . 13
3.4 | 5.4 Computing test set error . . . 14
3.5 | 5.5 Some questions, you have to answer... . . . 14

1 | 3 Regularized Linear Regression

1.1 | 3.1 Visualizing the dataset




```
Loading and Visualizing Data ...
```

1.2 | 3.2 Regularized linear regression cost function

```python
import numpy as np

def linearRegCostFunction(X, y, theta, Lambda):
    """computes the cost of using theta as the parameter for
       linear regression
    to fit the data points in X and y.
    Returns the cost in J
    """

    # Initialize some useful values

    m,n = X.shape # number of training examples
    theta = theta.reshape((n,1)) # in case where theta is a
       vector (n,)
    J = 0.

```

```
    # ====================== YOUR CODE HERE
      ======================
    # Instructions: Compute the cost of regularized linear
    #               regression for a particular choice of
      theta.
    #
    #               You should set J to the cost.
    #

    h = X @ theta
    e = h - y.reshape((m,1))

    J_error = np.sum(e**2) / (2*m)
    J_reg = (Lambda / (2*m)) * np.sum(theta[1:]**2)
    J = J_error + J_reg

    # =========================================================
    return J.flatten()
```

```
Loading and Visualizing Data ...
Cost at theta = [1  1]: 303.993192
(this value should be about 303.993192)
```

1.3 | 3.3 Regularized linear regression gradient

```
import numpy as np

def linearRegGradientFunction(X, y, theta, Lambda):
    """computes the gardients of the cost using theta as the
       parameter for linear regression
        to fit the data points in X and y.
        Returns the gradient in grad
     """

    # Initialize some values
    m,n = X.shape # number of training examples
    theta = theta.reshape((n,1)) # in case where theta is a
       vector (n,)
    grad = 0.

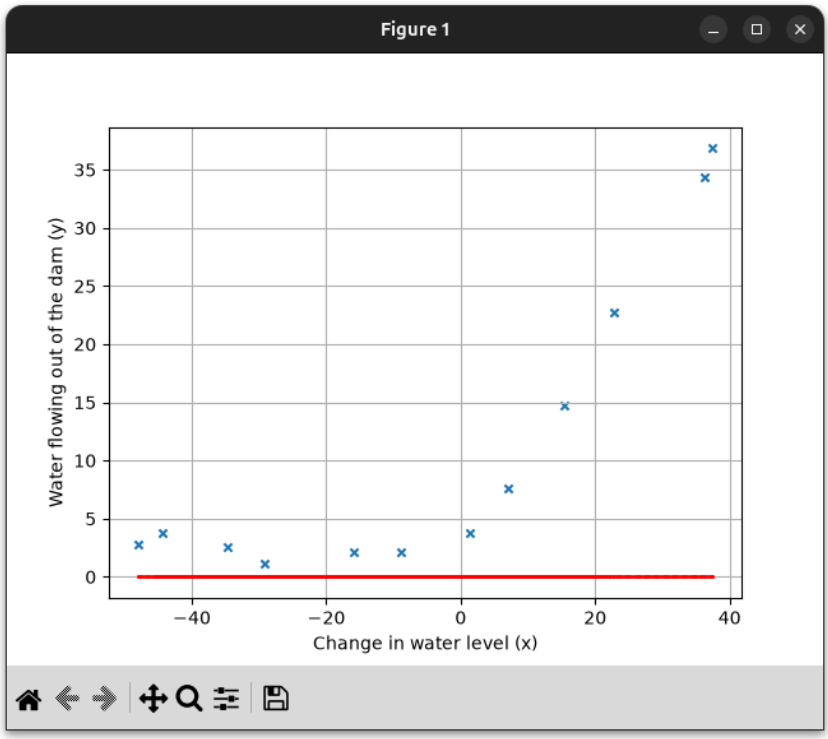
    # ====================== YOUR CODE HERE
       ======================
    # Instructions: Compute the gradient of regularized linear
    #               regression for a particular choice of
       theta.
    #
    #               You should set grad to the gradient.
    #

```

```
    h = X @ theta
    e = h - y.reshape((m,1))

    grad = (X.T @ e) / m
    grad[1:] += (Lambda / m) * theta[1:]

    # ============================================================
    return grad.flatten()
```




```
Gradient at theta = [1  1]:  [-15.303016 598.250744]
(this value should be about [-15.303016 598.250744])
```

1.4 | 3.4 Fitting linear regression

```
from unittest import result

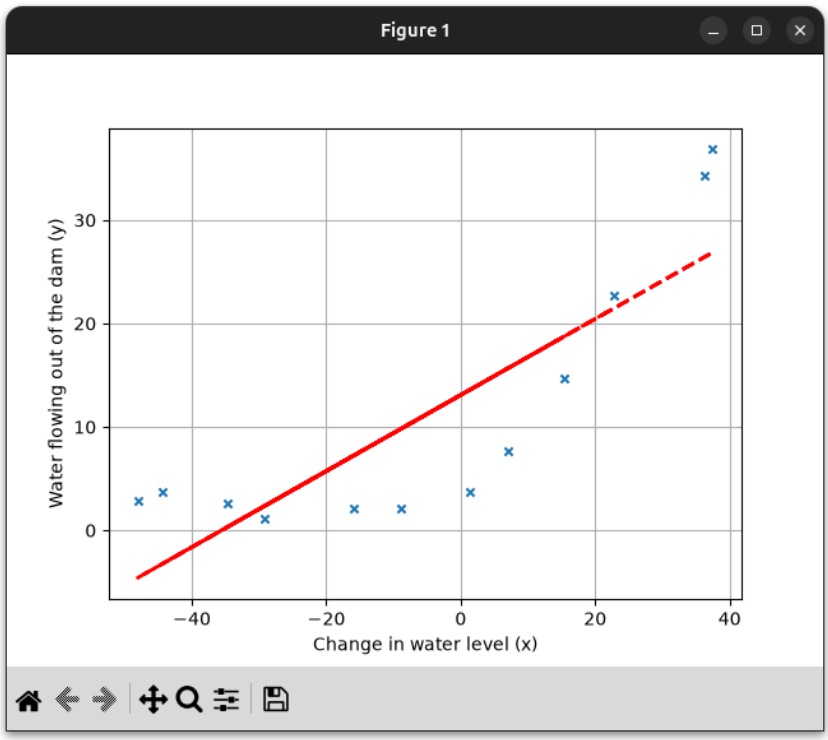
import numpy as np
from scipy.optimize import minimize

from linearRegCostFunction import linearRegCostFunction
from linearRegGradientFunction import
    linearRegGradientFunction


def trainLinearReg(X, y, Lambda, Xval=None, yval=None, maxiter
    =200):
```

```
11
12      """trains linear regression using
13      the dataset (X, y) and regularization parameter lambda.
           Returns the
14      trained parameters theta.
15      """
16
17      # Initialize Theta and costs history
18      theta = np.zeros(X.shape[1])
19      n = theta.size  # number of parameters
20      cost_train = []
21      cost_val = []
22
23      # fonction permettant a minimize de recuperer l'evolution
           des thetas
24      thetas = []
25      def save_step(*args):
26          for arg in args:
27              if type(arg) is np.ndarray:
28                  thetas.append(arg)
29
30      # Initialize Theta
31      initial_theta = np.zeros((X.shape[1]))
32
33      # Create "short hand" for the cost function to be
           minimized
34      costFunction = lambda t: linearRegCostFunction(X, y, t,
           Lambda)
35      gradFunction = lambda t: linearRegGradientFunction(X, y, t
           , Lambda)
36
37      # Descente de gradient
38      result = minimize(costFunction, initial_theta, method='CG'
           , jac=gradFunction, options={'disp': False, 'maxiter':
           maxiter, 'return_all':True}, callback=save_step)
39
40
41      # ====================== YOUR CODE HERE
           ======================
42      # Instructions: Compute
43      #   - theta the final thetas
44      #   - the training cost vs iteration number from thetas
45      #   - the validation cost vs iteration number from thetas
46
47      # outputs
48      theta = result.x
49
50      for t in thetas:
```

```
51          cost_train.append(linearRegCostFunction(X, y, t, 0)
                [0])
52
53      if Xval is not None and yval is not None:
54          for t in thetas:
55              cost_val.append(linearRegCostFunction(Xval, yval,
                    t, 0)[0])
56
57      # =========================================================
58
59      # force to np array
60      cost_train = np.array(cost_train)
61      cost_val = np.array(cost_val)
62
63      return theta, cost_train, cost_val
```



The training was successful, and the algorithm found the optimal parameters θ_0 and θ_1 that minimize the cost function.

However, the fitted line does not follow the data points well, especially at the extremes, where the data shows a nonlinear behavior.

This indicates underfitting (high bias) because the linear regression model is too simple to represent the data accurately.

2 | 4 Bias-variance

2.1 | 4.1 Learning curves

```
import matplotlib.pyplot as plt
from trainLinearReg import trainLinearReg

from trainLinearReg import trainLinearReg
from linearRegCostFunction import linearRegCostFunction
from linearRegGradientFunction import
   linearRegGradientFunction

def learningCurve(X, y, Lambda, Xval, yval, ax=plt.gca(),
   maxiter=2000):
    """returns the train and validation/test set costs for a
       learning curve. In particular,
    it returns two vectors of the same length - cost_train and
    cost_val.

    In this function, you will compute the train and
       validation costs.
    """

    # Number of training examples
    m, _ = X.shape
    cost_train = 0.
    cost_val = 0.

    # ====================== YOUR CODE HERE
       ======================
    # Instructions: Fill in this function to return training
       costs in
    #               cost_train and the validation errors in
       cost_val.
    #

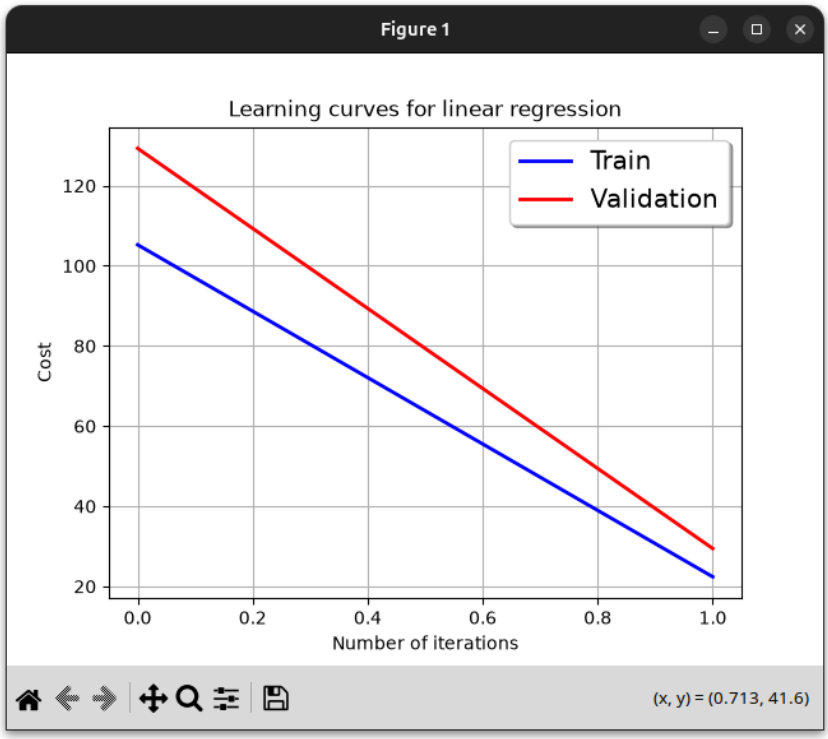
    # Erreur d'apprentissage et de generalisation
    _, cost_train, cost_val = trainLinearReg(X, y, Lambda,
       Xval, yval, maxiter)

    # =============================================

    # display
    plt.figure()
    ax.plot(range(cost_train.size), cost_train, color='b', lw
       =2, label='Train')
    ax.plot(range(cost_val.size), cost_val, color='r', lw=2,
       label='Validation')
    ax.set_title('Learning curves for linear regression')
```

```
    ax.grid()
    ax.set_xlabel('Number of iterations')
    ax.set_ylabel('Cost')
    ax.legend(loc='upper right', shadow=True, fontsize='x-
        large', numpoints=1)

    return cost_train, cost_val
```



Both training and validation costs decrease during training, showing that the model is learning.

However, both costs remain relatively high and close to each other.

This indicates underfitting (high bias) because the linear model is too simple to fit the data well.

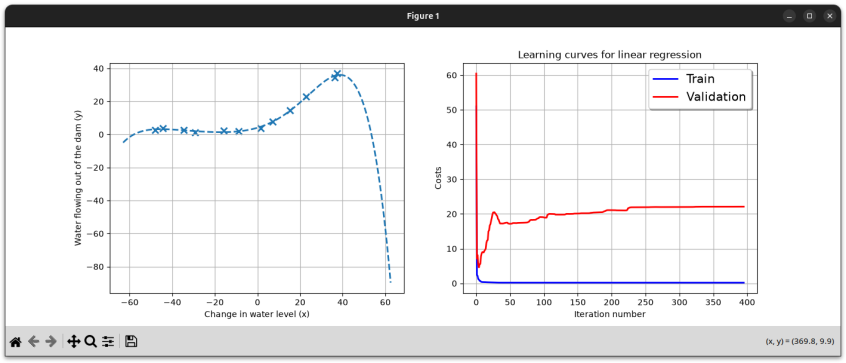
3 | 5 Polynomial regression

```
import numpy as np

def polyFeatures(X, p):
    """takes a data matrix X (size m x 1) and
    maps each example into its polynomial features where
    [X X**2 X**3 ...  X**p]
    """
    # You need to return the following variables correctly.
    X_poly = np.zeros((X.shape[0], p))
```

```
10
11      # ====================== YOUR CODE HERE
            ======================
12      # Instructions: Given a vector X, return a matrix X_poly
            where the p-th
13      #                 column of X contains the values of X to
            the p-th power.
14      #
15      #
16
17      for i in range(p):
18          X_poly[:, i] = X[:, 0] ** (i + 1)
19
20      # ============================================
21      return X_poly
```

3.1 | 5.1 Learning Polynomial Regression

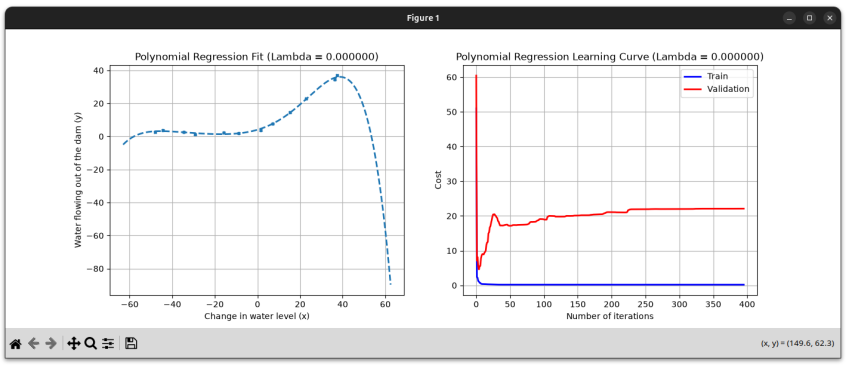


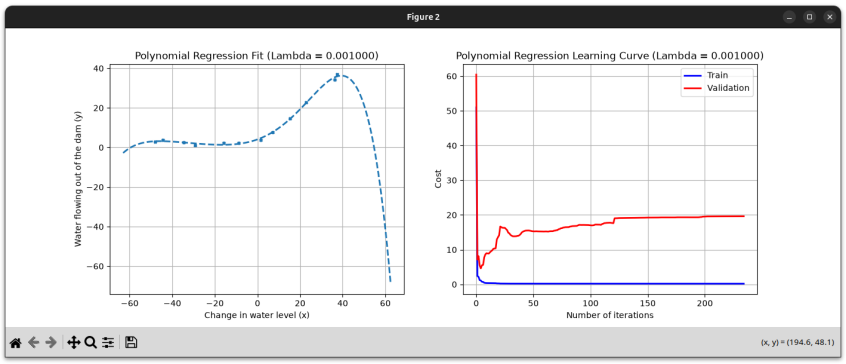
The polynomial model fits the training data well, but the curve changes too much at the extremes.

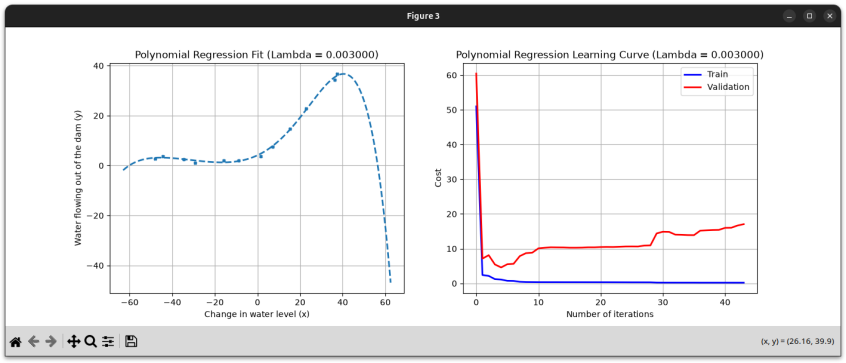
The training cost is very low, while the validation cost remains high.

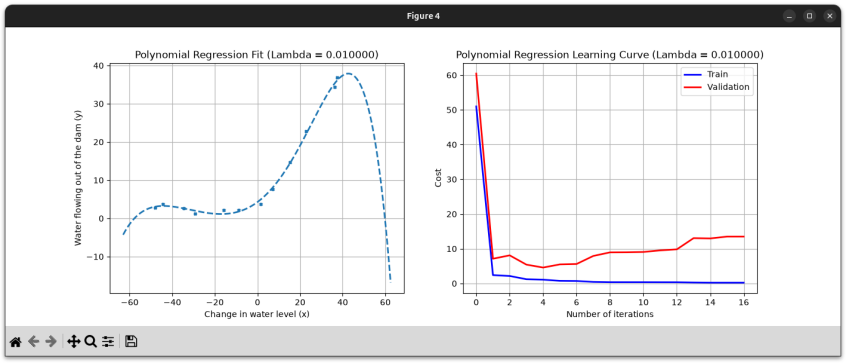
This indicates overfitting (high variance) because the model does not perform well on new data.

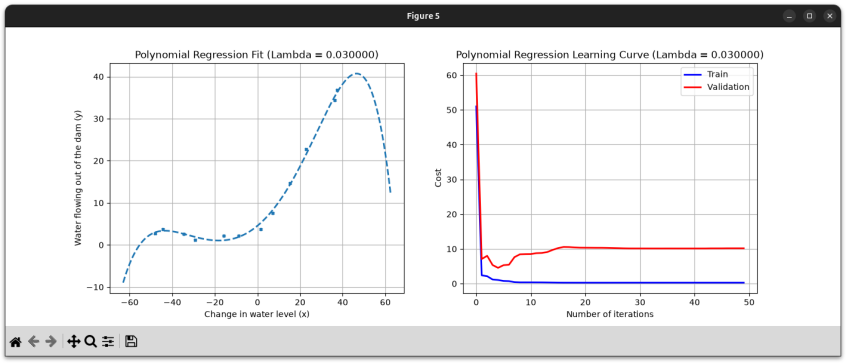
3.2 | 5.2 Adjusting the regularization hyper-parameter

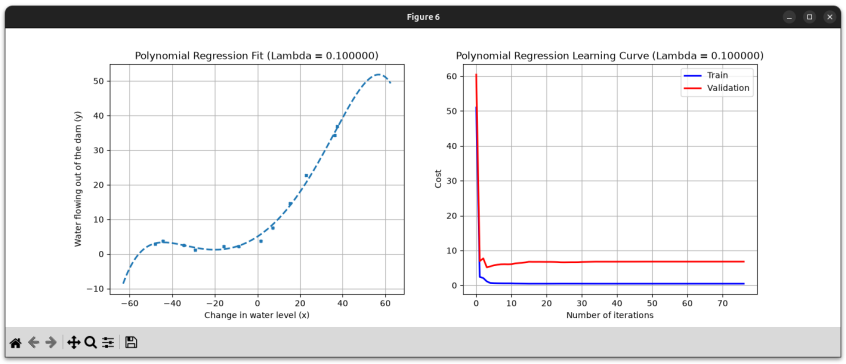


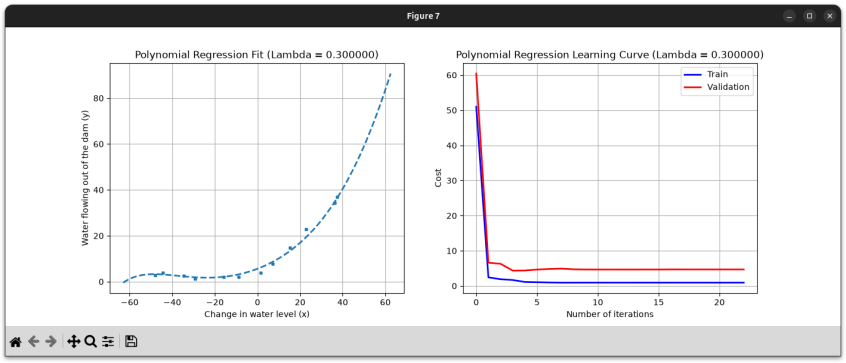


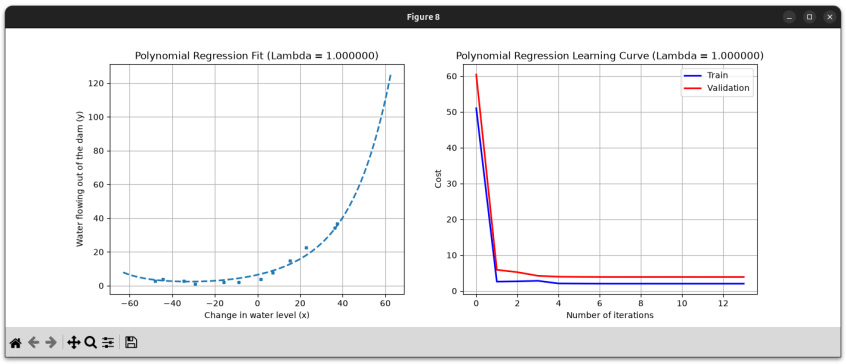


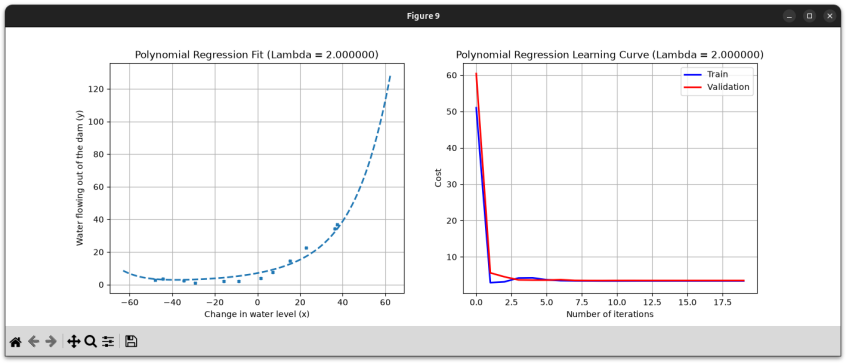


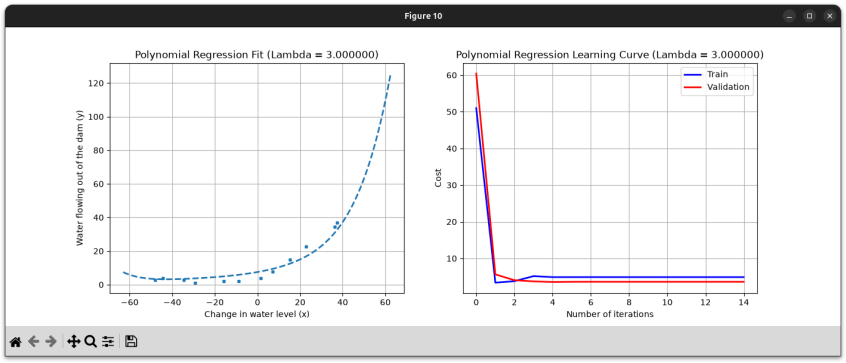

Figure 5
Polynomial Regression Fit (Lambda = 0.030000)
Water flowing out of the dam (y)
Change in water level (x)
Polynomial Regression Learning Curve (Lambda = 0.030000)
Train
Validation
Cost
Number of iterations

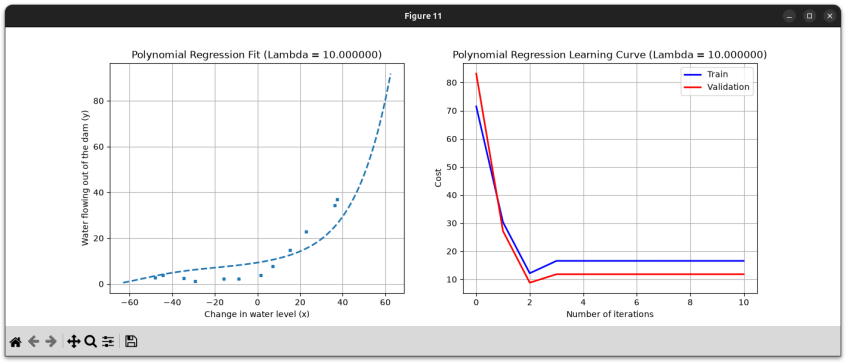

Figure 6
Polynomial Regression Fit (Lambda = 0.100000)
Water flowing out of the dam (y)
Change in water level (x)
Polynomial Regression Learning Curve (Lambda = 0.100000)
Train
Validation
Cost
Number of iterations

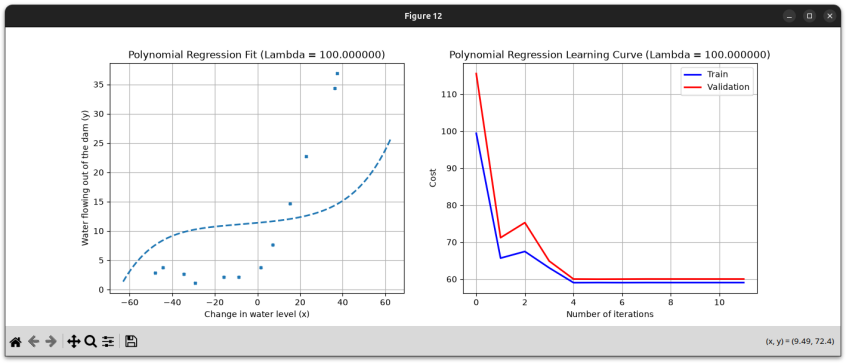

Figure 7
Polynomial Regression Fit (Lambda = 0.300000)
Water flowing out of the dam (y)
Change in water level (x)
Polynomial Regression Learning Curve (Lambda = 0.300000)
Train
Validation
Cost
Number of iterations


Figure 8
Polynomial Regression Fit (Lambda = 1.000000)
Water flowing out of the dam (y)
Change in water level (x)
Polynomial Regression Learning Curve (Lambda = 1.000000)
Train
Validation
Cost
Number of iterations









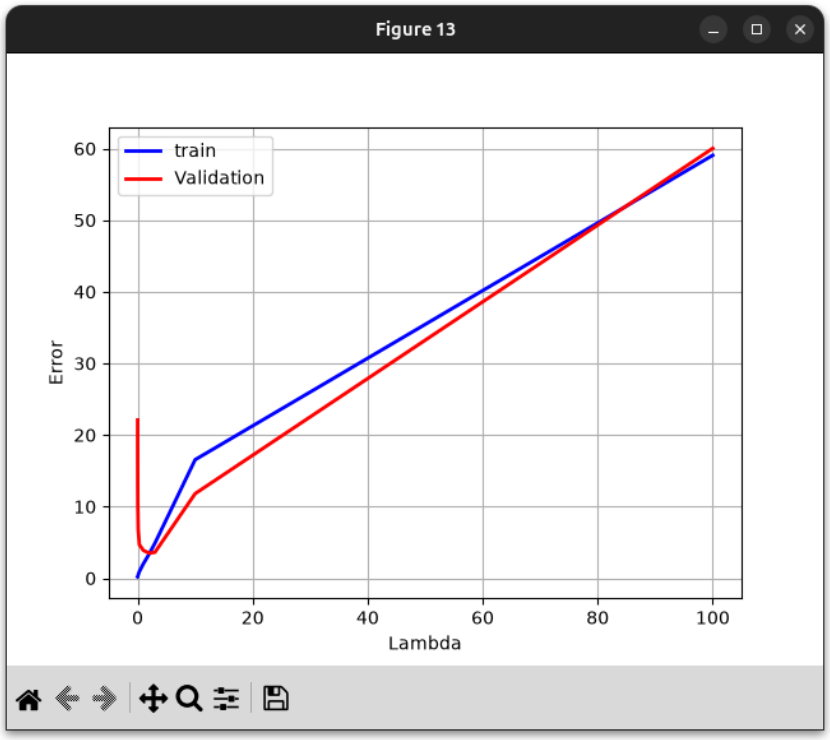
When $\lambda = 0$, the model fits the training data very well, but the validation cost is high. This indicates overfitting (high variance).

When $\lambda = 1$, the model fits the data well, and both costs are relatively low.

When $\lambda = 100$, the regularization is too strong, and both costs are high. This indicates underfitting (high bias).

Increasing λ reduces overfitting, but too much regularization causes underfitting.

3.3 | 5.3 Selecting λ using a validation set



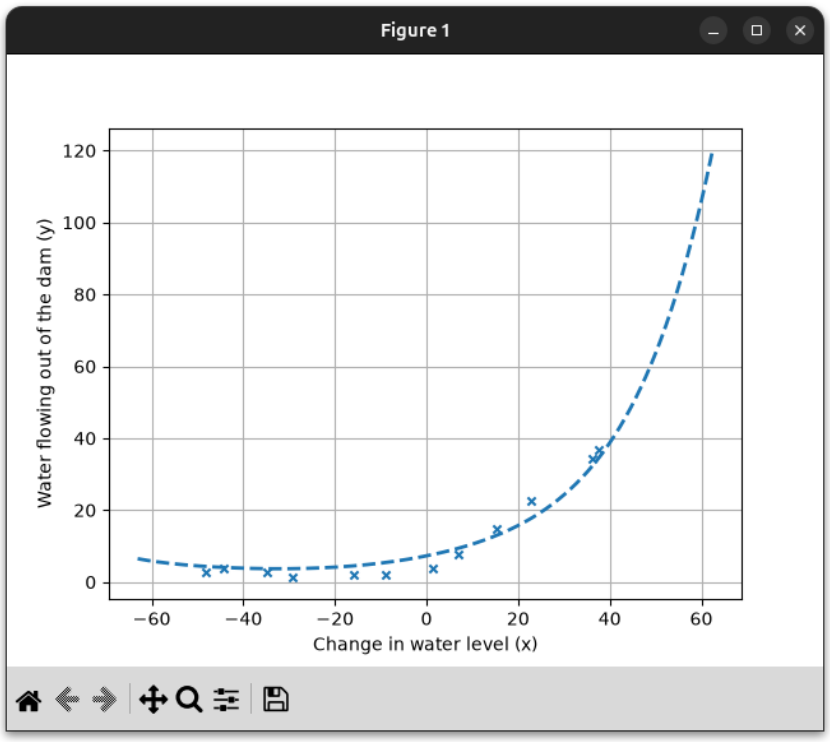
The validation cost is high when $\lambda = 0$, indicating overfitting.

As λ increases, the validation cost decreases. However, when λ is too high, both costs increase, indicating underfitting.

The best λ is the one with the lowest validation cost.

```
# ========== Part:  Computing test set error =============
Lambda = 2.0 # change to Lambda to the best value
```

3.4 | 5.4 Computing test set error




```
Best lambda: 2.0
Validation cost: 3.5034632316366903
test set error: 3.748681
```

The best value of λ is 2, with a validation cost of 3.5035.
The test cost is 3.7487, which is close to the validation cost.
This shows that the model fits the data well and gives good results on new data.

3.5 | 5.5 Some questions, you have to answer...

· Comment découpe-t-on un jeu de données pour entraîner et évaluer un modèle de prédiction?

R: On peut diviser les données en trois parties : les données d'entraînement (training set), de validation (validation set) et de test (test set).

· Définissez ce qu'est la capacité d'un modèle de prédiction (i.e. sa complexité). Sur quel ensemble l'évalue-t-on?

R: C'est la capacité du modèle à représenter les tendances des données, qu'elles soient simples ou complexes. On peut évaluer son ajustement avec les données d'entraînement.

· Définissez le concept d'erreur de généralisation? Sur quel ensemble l'évalue-t-on?

R: C'est l'erreur du modèle lorsqu'il fait des prédictions sur de nouvelles données. On l'évalue avec les données de test.

· A quoi sert l'ensemble de validation?

R: Il sert à comparer les résultats du modèle avec différents hyperparamètres, comme lambda, pour choisir celui qui donne la plus petite erreur de validation.

· Comment trouver les paramètres et fixer les hyperparamètres?

R: Les paramètres, comme theta, sont calculés pendant l'entraînement en minimisant la fonction de coût. Les hyperparamètres, comme lambda, sont choisis à partir des résultats de validation.

· A quoi sert la "learning curve"?

R: Elle sert à visualiser l'évolution des erreurs d'entraînement et de validation. Elle permet aussi d'identifier les problèmes d'underfitting et d'overfitting.